



VESTORA

ملف مذاكرة · مادة تحضير شخصية للمناقشة

Backend

المصادقة والحسابات، المشاريع والاكتشاف، القصة والتفاعل — تسعة كمنترولز وأربع خدمات

هذا الملف للمذاكرة فقط. ليس تقريرًا ولا وثيقة تُسلّم — الغرض منه أن تفهم جزءك من المشروع فهمًا كاملًا وتقدر تشرحه وتدافع عنه.

المحتويات

1. Your Responsibility, Exactly	4
1.1. The One-Line Answer	4
1.2. What You Own, in Detail	4
1.3. What's Not Yours	5
1.4. The Four Borders You Share	5
2. How the Backend Is Wired	6
2.1. The Shape: No Repository Layer	6
2.2. The Journey of One Request	6
2.3. Dependency Injection	6
3. Authentication — Your Deepest Chapter	7
3.1. Registration	7
3.2. Login — Five Gates in Order	8
3.3. Lockout — How It Actually Works	9
3.4. JWT — What's Inside the Token	9
3.5. Refresh Token Rotation — The Cleverest Part	10
3.6. The Security Log	10
3.7. Password Reset — Three Layers of Protection	11
3.8. The Development-Only Code Echo	11
4. Account Rules — The Mirrored File	12
4.1. Why It Exists in Two Places	12
4.2. The Rules Themselves	13
4.3. What It Replaced	13
4.4. How the Rules Reach the Endpoint	13
5. Users and Profiles	15
5.1. The Public Profile — One Rule, Two Places	15
5.2. One Round Trip Instead of Six	15
5.3. Account Deletion — Four Guards	15
5.4. Investor Fields Are Ignored, Not Rejected	16
5.5. Trust Signals — You Feed Them, You Don't Compute Them	16
6. Email Delivery	17
6.1. The Interface and the Implementation	17
6.2. Failure Is Reported, Not Swallowed	17
7. Projects — Your Largest Controller	18
7.1. The One Rule Everything Starts From	18
7.2. Visibility Is Viewer-Relative	19
7.3. Search — Two Different Behaviours by Length	19
7.4. Facets — Counts That Never Lie	20
7.5. Closing a Round — The Most Careful Code You Wrote	20
7.6. Rejected Listings Can Come Back	21
7.7. Images — Why They're Cached Hard	21
7.8. Soft Delete	21
8. Story, Signals, and Engagement	22

8.1. Project Story — Four Sections in One Controller	22
8.2. A Foreign-Key Detail Worth Knowing	23
8.3. Saved Searches — Computed on Read	23
8.4. Engagement — Honest Metrics	24
8.5. Bookmarks and Follows — Idempotent by Design	24
9. Endpoints Reference — All of Yours	25
9.1. Auth — <code>api/auth</code>	25
9.2. Users — <code>api/users</code>	25
9.3. Projects — <code>api/projects</code>	26
9.4. Story, Signals, Engagement	26
10. Request and Response Examples	27
10.1. Register	27
10.2. Login	28
10.3. Browse Feed	28
10.4. A Facet Response	29
11. Authorization — Role Is Not Ownership	29
11.1. The Rule	29
11.2. The Three Levels in Your Code	29
12. The Database Boundary	30
12.1. You Consume the Schema, You Don't Change It	30
12.2. What You Do Touch	31
13. Known Limitations in Your Area	31
14. Doctor Questions and Pressure Follow-ups	32
15. How to Demo Your Part Live	34
15.1. Before You Start	34
15.2. The Five-Minute Scenario	34
16. Terms You Must Know	35
17. Suggested Study Plan	36
18. Final Code Map — Where to Find Everything	36
19. Cheat Sheet — Review Before You Walk In	38

1. Your Responsibility, Exactly

1.1. The One-Line Answer

إنت مسؤول عن باب الدخول للمنصة وقلب محتواها: المصادقة والحسابات من ناحية، والمشاريع والاكتشاف والتفاعل من الناحية الثانية.

اعرف ده كويس

الجملة اللي تقولها لو اتسألت «إنت عملت إيه في الباكند؟»:

«أنا مسؤول عن المصادقة والحسابات كاملة — التسجيل، الدخول، الـ JWT وتدوير الـ refresh tokens، تأكيد الإيميل، إعادة تعيين كلمة السر، قفل الحساب — وعن المشاريع والاكتشاف: إنشاء المشروع وتعديله، البحث والفلاتر والعدّادات، القصة والتحديثات، والتفاعل والمتابعة.»

دي جملة محدّدة وقابلة للإثبات، مش «شغلت على الباكند».

1.2. What You Own, in Detail

المجال	الأسطر	الملفات
المصادقة والحسابات	1161	AccountRules · IAuthService · AuthService · AuthController
المستخدمين والبروفایل	552	UsersController
المشاريع والاكتشاف	2040	ProjectStoryController · ProjectsController · SignalsController
التفاعل والمتابعة	765	BookmarkController · ProjectEngagementController · FeedController · FollowController
الإيميل	99	IService · MailKitEmailService

جدول 1 نصيبك: 9 كترولرز و4 ملفات خدمات ≈ 4675 سطر.

اعرف ده كويس

الرقم اللي تحفظه: **4675 سطر**، تقريبًا ثلث الباكند كله. ده رقم محدّد ومحسوب، ولو الدكتور سأل «قد إيه شغلك؟» الرقم أحسن من «كثير».

1.3. What's Not Yours

ميش جزءك

- الدفع والتمويل كله — `PaymentService` · `FundingMath` · Stripe · `webhooks` · `sweeper`. قائد الفريق.
 - غرفة الصفقة والتفاوض — `DealRoomController` · `TermSheetService`. قائد الفريق.
 - لوحات المؤسّس والمستثمر — `InvestorController` · `FounderDashboardController` · `CapitalController`. قائد الفريق.
 - الرسائل والإشعارات و SignalR — `MessagesController` · `NotificationController` · `ChatHub`. قائد الفريق.
 - الأدمن والمراجعة والإيرادات — كل الـ `Admin*` و `ReportController`. قائد الفريق.
 - `TrustSignals` و `StageLog` و `PipelineStages` — قائد الفريق.
 - البنية التحتية — `Program.cs` · تسجيل الـ DI · `ServiceResult` · `UtcDateTimeConverter` · `FileUploadSecurityService`. قائد الفريق.
 - الكيانات والترحيلات (migrations) — قائد الفريق. إنت مبتلمسش السكيما خالص.
- لو اتسألت في أي حاجة من دول: «دي بتاعة قائد الفريق» — إجابة محترمة، وأحسن بكثير من محاولة تدّعي معرفة أول سؤال متابعة هيكشفها.

1.4. The Four Borders You Share

دي أخطر حّتة في المناقشة، لأنها الحّتة اللي فيها كودك بيلمس كود غيرك.

إيه اللي بيحصل عنده	الحّد
قواعد التحقق مكرّرة في الباكند (بتاعتك) والفرونت (بتاعة القائد). لازم يفضلوا مطابقين — القاعدة مكتوبة في <code>AGENTS.md</code> .	<code>AccountRules.cs</code> ↔ <code>rules.ts</code>
<code>UsersController</code> و <code>ProjectsController</code> بتوعك بينادوا عليه. إنت بتعرض الإشارات، القائد بيحسبها.	<code>TrustSignals</code> (بتاع القائد)
<code>AuthService</code> بتاعك بينادي <code>MailKitEmailService</code> بتاعك — ده الحّد الوحيد اللي جوّه نصيبك بالكامل.	<code>IService</code>
محتاج عمود جديد؟ بتطلب من القائد يعمل الـ entity والـ migration. إنت بتستهلك السكيما، ميش بتغيّرها.	الداتا بيز

جدول 2 أربع حدود بين كودك وكود غيرك.

اختبر نفسك

1. إيه دورك في الباكند بالطبط؟ المصادقة والحسابات كاملة، والمشاريع والاكتشاف والقصة والتفاعل — 9 كينترولز و 4 خدمات، حوالي 4675 سطر.
2. الدفع بتاع مين؟ قائد الفريق — `PaymentService` و `FundingMath` وكل ما يتعلق بـ Stripe.
3. لو محتاج عمود جديد في الداتا بيز تعمل إيه؟ أطلب من القائد يعمل الـ entity والـ migration — أنا مبلمسش السكيما.
4. ليه `AccountRules` موجودة في الباكند والفرونت؟ الفرونت عشان المستخدم يعرف قبل ما بيعت؛ الباكند هو اللي بيقرّر، لأن الـ endpoints بتتنده من غير النموذج أصلاً.

2. How the Backend Is Wired

قبل أي فيتشر، لازم تعرف الطلب بيمشي إزاي من أول ما يوصل لحد ما يرجع.

2.1. The Shape: No Repository Layer

اعرف ده كويس

في المشروع ده الكنترولر بيكلم `AppDbContext` مباشرة. مفيش طبقة Repository ولا Unit of Work ولا Clean Architecture.

ودي مش «حاجة ناقصة» — دي قرار. لو الدكتور سأل «ليه مفيش Repository؟» الإجابة: `AppDbContext` نفسه أصلاً Repository و Unit of Work — إضافة طبقة فوقه كانت هتبقى غلاف بينده على الغلاف اللي تحته من غير ما يضيف حاجة.

الخدمات موجودة بس لما يكون فيه منطق حقيقي مشترك — زي `AuthService` بتاعك.

2.2. The Journey of One Request

خد `POST /api/auth/register` كمثال، وده أوضح مسار في نصيبك:

الخطوة	اللي يحصل
1. Routing	<code>[Route("api/[controller]")]</code> بيوصل الطلب لـ <code>AuthController</code> ، و <code>[HttpPost("register")]</code> بيحدد الـ action.
2. Model Binding	الـ JSON بيتحول لـ <code>RegisterDto</code> . وهنا بيحصل التحقق — الـ attributes زي <code>[PersonName]</code> و <code>[PasswordPolicy]</code> بتشتغل تلقائيًا.
3. ModelState	لو التحقق فشل، الرد بيرجع 400 بأخطاء على مستوى كل حقل — عشان الفرونت يعرف يحط كل رسالة جنب حقلها.
4. الكنترولر	بينده <code>_authService.RegisterAsync(dto)</code> — الكنترولر هنا ربيع جدًا، المنطق كله في الخدمة.
5. الخدمة	<code>AuthService</code> بيعمل كل حاجة: يتأكد الإيميل مش موجود، يعمل هاش لكلمة السر، يولد كود التحقق، يسجل حدث أمني، يحفظ، بيعت إيميل.
6. <code>ServiceResult</code>	الخدمة بترجع نتيجة فيها الحالة (Ok / BadRequest / Unauthorized)، والكنترولر بيحولها لرد HTTP.

جدول 3 رحلة طلب التسجيل من أوله لآخره.

اعرف ده كويس

لاحظ الفصل: الكنترولر بيترجع HTTP، والخدمة بتاخذ القرار. عشان كده `AuthController` 153 سطر بس بينما `AuthService` 723 سطر.

ودي إجابة قوية لو الدكتور سأل «ليه فيه خدمة للمصادقة ومفيش للمشاريع؟» — لأن المصادقة فيها منطق مشترك بين حاجات كتير (توليد التوكن، تسجيل الأحداث، إلغاء الجلسات)، والمشاريع منطقها مربوط بالـ endpoint نفسه.

2.3. Dependency Injection

`AuthService` بياخد تسع حاجات في الـ constructor:

```

1 public AuthService(
2     ApplicationDbContext dbContext,
3     IConfiguration configuration,
4     IEmailService emailService,
5     IFileUploadSecurityService fileUploadSecurityService,
6     IOptions<AuthSecuritySettings> securitySettings,
7     IOptions<EmailSettings> emailSettings,
8     IHostEnvironment environment,
9     IHttpContextAccessor httpContextAccessor,
10    ILogger<AuthService> logger)

```

اعرف ده كويس

إنت مبتعملش `new AuthService(...)` في أي مكان — الـ `container` بيعملها. إنت بتطلب الحاجة كـ `interface` والـ `container` بيحيلك التنفيذ.

فايدة عملية واضحة: `IEmailService` ممكن يبقى `MailKitEmailService` الحقيقي، أو أي تنفيذ ثاني، من غير ما `AuthService` يتغير ولا سطر.

تسجيل الخدمات نفسه في `Program.cs` — ده بتاع القائد.

خد بالك

`IOptions<AuthSecuritySettings>` مش نفس `IConfiguration`. الـ `Options` بتوصل لك كائن مكتوب بأنواع (typed) بدل قراءة نص من ملف الإعدادات كل مرة. عشان كده كل أرقام الأمان — مدة القفل، عدد المحاولات، صلاحية التوكن — بتيجي من `_securitySettings` مش من قيم متناثرة في الكود.

اختبر نفسك

1. فين المنطق: في الكترولر ولا الخدمة؟

الكترولر بيترجم HTTP بس؛ المنطق في الخدمة. عشان كده `AuthService` 153 سطر و `AuthController` 723.

2. ليه مغيث طبقة Repository؟

`AppDbContext` نفسه Repository و Unit of Work — طبقة فوقه كانت هتبقى غلاف على غلاف.

3. إمتى التحقق بيحصل في رحلة الطلب؟

في خطوة الـ Model Binding، عن طريق الـ attributes قبل ما الكود بتاعي يشتغل أصلاً.

3. Authentication — Your Deepest Chapter

`AuthService` 723 سطر، وهو أكثر ملف هتتسأل فيه. الفصل ده بياخده حصة حصة.

3.1. Registration

الخطوة	التفصيل
تطبيع الإيميل	<code>NormalizeEmail</code> — <code>Trim()</code> و <code>ToLowerInvariant()</code> ، عشان الإيميل بحروف كبيرة ونفسه بحروف صغيرة يبقوا نفس الحساب.
فحص التكرار	<code>AnyAsync(u => u.Email.ToLower() == normalizedEmail)</code> قبل أي حاجة.

الخطوة	التفصيل
تحديد النوع	<code>switch</code> على <code> UserType : investor</code> بتعمل <code>new Investor</code> ، <code>innovator</code> بتعمل <code>new Innovator</code> . أي حاجة ثانية = رفض.
تنظيف الاسم	<code>Regex.Replace</code> بتجمع المسافات المتكررة، عشان الاسم ميتخزنش بمسافات زيادة أو بسطر جديد اتلرق معاه.
هاش كلمة السر	<code>BCrypt.Net.BCrypt.HashPassword(dto.Password)</code> — كلمة السر ميتخزنش أبدًا كنص.
كود التحقق	رقم من 6 خانات، بيتخزن مهيئًا مع وقت انتهاء.
حدث أمني	<code>AddSecurityEvent</code> بنوع <code>register</code> قبل الحفظ.
الإيميل	<code>TrySendVerificationEmailAsync</code> — بعد الحفظ.

جدول 4 خطوات RegisterAsync بالترتيب.

اعرف ده كويس

أهم تفصيلة في التسجيل كله — الحساب بيتعمل حتى لو الإيميل مانفعش يتبع، والرسالة بتتغير حسب اللي حصل فعلاً:

```

1 var delivered = await TrySendVerificationEmailAsync(user, verificationToken);
2 return ServiceResult<ApiMessageDto>.Created(new ApiMessageDto
3 {
4     Message = delivered
5         ? "Registration completed successfully. Please verify your email."
6         : "Your account was created, but we could not send the verification
code..."
7 });

```

التعليق في الكود بيشرح ليه: ضياع تسجيل صحيح لأن مزود البريد كان مقطوع أسوأ من حساب غير مؤكد يقدر يعيد الإرسال منه — بس الرسالة لازم تطابق اللي حصل فعلاً. دي إجابة ممتازة لو الدكتور سأل عن أصعب قرار خدته.

3.2. Login — Five Gates in Order

الترتيب هنا **مقصود**، والدكتور ممكن يسأل عنه:

البوابة	الرد لو فشلت
1. المستخدم موجود؟	رسالة عامة : إيميل أو كلمة سر غير صحيحة.
2. الحساب مقفول؟	رسالة إن الحساب مقفول مؤقتًا.
3. الحساب موقوف بأمر أدمن؟	رسالة إن الحساب موقوف والتواصل مع الدعم.
4. كلمة السر صح؟	<code>RegisterFailedLogin(user)</code> + نفس الرسالة العامة بتاعة البوابة 1.
5. الإيميل مؤكد؟	رسالة إن لازم يأكّد إيميله الأول.

جدول 5 بوابات `LoginAsync` بالترتيب.

اعرف ده كويس

ليه البوابة 1 و 4 بيرجّعوا نفس الرسالة بالظبط؟

عشان محدش يقدر يعرف إذا كان الإيميل مسجّل عندنا ولا لا. لو قلنا الإيميل ده مش موجود في الحالة الأولى، أي حد يقدر يجرب قائمة إيميلات ويعرف مين عضو في المنصة — ودي تسريب معلومات، اسمها **user enumeration**. ده قرار أمني مقصود، مش كسل في الرسائل. واحدة من أقوى الحاجات اللي تقولها في المناقشة.

خد بالك

لاحظ إن البوابة 2 و 3 بيرجّعوا رسائل مختلفة — وده مش تناقض مع اللي فوق. الفرق: القفل والإيقاف بيحصلوا لحساب إحنا متأكدين إنه موجود. إخفاء السبب هنا كان هيجلي المستخدم الحقيقي تايه من غير أي فائدة أمنية.

3.3. Lockout — How It Actually Works

```

1 private void RegisterFailedLogin(User user)
2 {
3     user.FailedLoginCount++;
4     user.LastFailedLoginAtUtc = DateTime.UtcNow;
5
6     if (user.FailedLoginCount >= _securitySettings.MaxFailedLoginAttempts)
7     {
8         user.LockoutEndUtc =
9             DateTime.UtcNow.AddMinutes(_securitySettings.LockoutMinutes);
10        AddSecurityEvent("account_locked", user, user.Email, ...);
11    }
12 }
```

اعرف ده كويس

IsLockedOut فيها تفصيلة ذكية: هي مش بس بتقرا، هي كمان بتنصف.

```

1 if (user.LockoutEndUtc > DateTime.UtcNow) return true;
2 user.LockoutEndUtc = null;           // القفل خلس
3 user.FailedLoginCount = 0;          // العداد يرجع صفر
4 return false;
```

يعني مفيش مهمة خلفية بتفكّ الأقفال — القفل بيفكّ نفسه أول ما حد يحاول يدخل بعد انتهاء المدة. أبسط وأرخص من job بيلفّ على كل الحسابات.

3.4. JWT — What's Inside the Token

```

1 var claims = new List<Claim>
2 {
3     new(ClaimTypes.NameIdentifier, user.Id.ToString()),
4     new(ClaimTypes.Email, user.Email),
5     new(ClaimTypes.Name, user.UserName),
6     new(ClaimTypes.Role, user.UserType),
7     new("UserType", user.UserType),
8     new("email_verified", user.IsEmailVerified ? "true" : "false")
9 };

```

اعرف ده كويس

الـ `ClaimTypes.Role` هو اللي بيخلى `[Authorize(Roles = "Innovator")]` تشتغل. يعني الدور جوّه التوكن نفسه — السيرفر مش بيروح للداتا بيز كل طلب عشان يعرف إنت مؤسس ولا مستثمر. وده بيفسّر حاجة مهمة: لو دور المستخدم اتغيّر، التوكن القديم بيفضل شايل الدور القديم لحد ما ينتهي. عشان كده مدة التوكن قصيرة.

خد بالك

فيه فحصان على مفتاح التوقيع قبل ما أي توكن يتولّد: إن المفتاح موجود، وإنه 32 بايت على الأقل. المفتاح القصير معناه توكن يقدر حد يزوره، والرفض هنا مقصود ولازم يفضل.

3.5. Refresh Token Rotation — The Cleverest Part

وذي أذكى حتّة في نصيبك كله. لو الدكتور سأل عن أصعب حاجة عملتها — دي.

القرار	السبب
التوكن بيتخزّن مهشّرًا	<code>TokenHash = HashToken(rawToken)</code> — لو الداتا بيز اتسرّبت، التوكنات نفسها مش موجودة فيها.
كل استعمال بيولّد توكن جديد	التوكن القديم بياخد <code>RevokedAtUtc</code> و <code>ReplacedByTokenHash</code> .
64 بايت عشوائي	<code>RandomNumberGenerator.GetBytes(64)</code> — مش <code>Guid</code> ولا <code>Random</code> ، لأن دول متوقّعين نسبيًا.

جدول 6 قرارات تصميم الـ refresh token.

اعرف ده كويس

كشف إعادة الاستخدام (reuse detection) — أهم كام سطر في الملف:

```

1 if (!existingToken.IsActive)
2 {
3     if (existingToken.IsRevoked)
4     {
5         await _dbContext.Entry(existingToken.User)
6             .Collection(u => u.RefreshTokens).LoadAsync();
7         RevokeActiveRefreshTokens(existingToken.User, "Refresh token reuse
detected.");
8     }
9 }

```

السيناريو اللي بنحمله: حد سرق توكن. المستخدم الحقيقي استعمله فاندور لتوكن جديد والقديم اتلغى. بعدين السارق يجي يستعمل التوكن القديم المُلغى — وساعتها إحنا عارفين إن فيه نسخة مسروقة، فبنلغي كل جلسات الحساب. المستخدم بيتطرد ويسجّل دخول تاني، والسارق بيفضل بلا حاجة. دي أقوى إجابة عندك في الملف كله. احفظ السيناريو ده بالنص.

3.6. The Security Log

كل عملية في `AuthService` بتكتب سطر في `SecurityLogs`:

نوع الحدث	إمتى بيتسجّل
register	تسجيل جديد

نوع الحدث	إمتى بيتسجل
login_success · login_failed	نجاح أو فشل الدخول
login_blocked_lockout	محاولة دخول لحساب مقفول
login_blocked_suspended	محاولة دخول لحساب موقوف
login_blocked_unverified_email	إيميل مش مؤكّد
account_locked	الحساب انتقل بعد محاولات فاشلة
refresh_token_rotated · _revoked · _failed	دورة حياة التوكن
password_reset_completed · password_changed	تغيير كلمة السر
email_verification_resent · email_verified	تأكيد الإيميل

جدول 7 أنواع الأحداث اللي كودك بيتسجلها.

خد بالك

الجدول ده هو اللي بيغذي شاشة الأمان في لوحة الأدمن — والشاشة دي بتاعة القائد، بس البيانات اللي فيها بتيجي من كودك إنت.

لو الدكتور فتح شاشة الأمان وسأل الأرقام دي جاية منين — الإجابة: `AddSecurityEvent` في `AuthService`.

3.7. Password Reset — Three Layers of Protection

الحماية	التفصيل
رسالة موحّدة	نفس الرسالة بترجع سواء الإيميل موجود أو لا — نفس منطق منع تسريب المستخدمين.
فترة تهدئة	<code>PasswordResetResendCooldownSeconds</code> — بتمنع إغراق حد برسائل.
حدّ للمحاولات	<code>PasswordResetFailedAttempts</code> — بعد عدد معيّن الكود بيتمسح خالص عن طريق <code>ClearPasswordReset</code> .

جدول 8 ثلاث طبقات على إعادة التعيين.

اعرف ده كويس

وفيه تفصيلة مشتركة بين تغيير كلمة السر وإعادة تعيينها:

```
1 RevokeActiveRefreshTokens(user, "Password changed.");
```

يعني لما كلمة السر تتغير، كل الجلسات المفتوحة بتتلغي. السبب واضح: لو حد كان داخل على حسابك، تغيير كلمة السر لازم يطرده — مش يسيبه شغال بتوكن قديم.

3.8. The Development-Only Code Echo

```
1 private void EchoCodeForDevelopment(string purpose, User user, string code)
2 {
3     if (!emailSettings.LogCodesInDevelopment || !_environment.IsDevelopment())
4         return;
5     ...
6 }
```

اعرف ده كويس

لاحظ إن فيه **فحصين** مش واحد: إعداد صريح و بيئة تطوير.

التعليق في الكود بيشرح ليه: كود تحقق في سجلات الإنتاج معناه اختطاف حساب لأي حد يقدر يقرأ السجلات. فحص واحد مش كفاية — قيمة إعداد غلط ماينفعش تشغلها في الإنتاج، والاشتغال في وضع التطوير ماينفعش يشغلها بالمفاجأة.

مثال ممتاز على الدفاع بطبقتين لو اتسألت عن الأمان.

فين ده في الكود

```
Services/AuthService.cs
AuthService.RegisterAsync
AuthService.LoginAsync
AuthService.RefreshTokenAsync
AuthService.RegisterFailedLogin
AuthService.AddSecurityEvent
Controllers/AuthController.cs
```

كل منطق المصادقة — 723 سطر
التسجيل وإرسال كود التحقق
البوابات الخمسة بالترتيب
التدوير وكشف إعادة الاستخدام
عداد المحاولات والقفل
تسجيل كل حدث أمني
تسع نقاط نهاية رقيقة بتنده الخدمة

اختبر نفسك

1. ليه رسالة «إيميل أو كلمة سر غلط» عامة؟
عشان محدش يقدر يعرف مين مسجل في المنصة — منع تسريب المستخدمين (user enumeration).
2. إيه اللي بيحصل لو حد استعمل refresh token مُلغى؟
باعتبرها سرقة ونلغي كل جلسات الحساب — كشف إعادة الاستخدام.
3. كلمة السر متخزنة إزاي؟
هاش بـ BCrypt، مش نص. `HashPassword` عند الحفظ و `Verify` عند الدخول.
4. ليه الحساب بيتعمل حتى لو الإيميل مانفعش يتبع؟
عشان ضياع تسجيل صحيح بسبب مزود بريد مقطوع أسوأ من حساب غير مؤكد — بس الرسالة بتقول الحقيقة.
5. القفل بيتفك إزاي؟
`IsLockedOut` بتنصفه بنفسها أول محاولة دخول بعد انتهاء المدة — مفيش مهمة خلفية.

4. Account Rules — The Mirrored File

ملف صغير (258 سطر) بس هو **أخطر حد مشترك** عندك، لأنه مكرّر في الفرونـت. `AccountRules.cs`

4.1. Why It Exists in Two Places

اعرف ده كويس

التعليق في أول الملف بيشرحها بجملة واحدة:

«النسخة اللي في الفرونـت موجودة عشان حد يتقال له قبل ما بيعت؛ والنسخة دي هي اللي بتقرّر، لأن الـ endpoints بتتوصل من غير النموذج أصلاً.»

يعني: الفرونـت **راحة استخدام**، الباكند **أمان**. حد يقدر بيعت طلب مباشر بـ Postman من غير ما يفتح الصفحة خالص — ولو التحقق كان في الفرونـت بس، كان هيعدي.

دي أوضح إجابة لو الدكتور سأل «ليه بتتحقق مرتين؟ مش ده تكرر؟»

خد بالك

القاعدة مكتوبة في `AGENTS.md`: أي تغيير في `AccountRules.cs` لازم يتعمل في `rules.ts` كمان والعكس. لو غيرت واحدة بس، بيحصل موقف سخيف: المستخدم يعدّي من الفرونت ويبجي السيرفر يرفضه، أو العكس.

4.2. The Rules Themselves

القاعدة	القيمة	التفصيل
طول كلمة السر	10 على الأقل	<code>PasswordMinLength = 10</code>
أقصى طول	72 بايت	حدّ Bcrypt نفسه — أي حاجة أطول بتتقصّ بصمت.
تركيب كلمة السر	—	حرف كبير + حرف صغير + رقم، ومفيش مسافات في الأول أو الآخر.
كلمات شائعة	قائمة مرفوضة	<code>CommonPasswords</code> — فيها <code>vestora123</code> و <code>investor123</code> كمان.
كلمة السر ≠ الإيميل	—	لو الجزء اللي قبل @ طوله 4 أحرف أو أكثر وموجود في كلمة السر، ترفض.
الاسم	2-50 حرف	بعد ال trim، ولازم يحتوي حرف — عشان العربي والإنجليزي يعدّوا «و» و«12» يترفضوا.
الإيميل	254 حرف	شكل بسيط ب regex — مش تحقّق كامل من RFC.
الهاتف	7-15 رقم	مفيش شكل قومي مفروض — فيستورا بتأخذ أعضاء من أي مكان.
تاريخ الميلاد	18-100 سنة	بحساب عمر حقيقي، ومش في المستقبل.

جدول 9 كل قواعد `AccountRules` بالأرقام.

اعرف ده كويس

أذكر قاعدة في الملف — حدّ ال 72 بايت:

التعليق بيقل: «Bcrypt بيتجاهل أي حاجة بعد 72 بايت. قبول كلمة سر أطول معناه إننا بنخزّن جزء منها بصمت — وبعدين أي نص ثاني طويل بنفس أول 72 بايت هيفتح الحساب كمان. الرفض هو الأمين.»

دي إجابة ممتازة جدّا لو الدكتور سأل «ليه فيه حدّ أقصى لكلمة السر؟» — معظم الناس بتفتكر إن الحدّ الأقصى ضعف، وهنا هو أمان.

4.3. What It Replaced

خد بالك

التعليق بيوتّق اللي كان قبله — وده مفيد جدّا في المناقشة:

- التسجيل كان بيقبل كلمة سر من 6 أحرف من غير أي شروط، يعني `123456` كان اعتماد صالح على منصّة أعضاؤها بينشروا مستندات تمويل.
 - إعادة التعيين كانت بتطلب 8، وتغيير كلمة السر بيطلب 8 — ثلاث حدود مختلفة لنفس السر.
 - تاريخ الميلاد مكانش عليه أي حدود، فسنة 3000 كانت بتسجّل.
 - الأسماء كانت بتتقاس قبل ال trim، فمسافتين كانوا اسم عيلة صالح.
- لو اتسألت «إيه المشكلة اللي حلّيتها؟» — دي إجابة بأمثلة محدّدة.

4.4. How the Rules Reach the Endpoint

مش ببناء يدوي — ب `attributes` بتشتغل وقت ال model binding:

```

1 public class RegisterDto
2 {
3     [Required, PersonName]
4     public string FirstName { get; set; } = string.Empty;
5
6     [Required, EmailAddress, StringLength(AccountRules.EmailMaxLength)]
7     public string Email { get; set; } = string.Empty;
8
9     [Required, PasswordPolicy(EmailProperty = nameof(Email))]
10    public string Password { get; set; } = string.Empty;
11
12    [Required, Compare("Password")]
13    public string ConfirmPassword { get; set; } = string.Empty;
14
15    [Required, RealisticBirthDate]
16    public DateTime BirthDate { get; set; }
17
18    [Required, DialablePhone]
19    public string Phone { get; set; } = string.Empty;
20 }

```

اعرف ده كويس

- في آخر `AccountRules.cs` فيه أربع `ValidationAttribute` مخصصة: `PasswordPolicyAttribute`، `RealisticBirthDateAttribute`، `DialablePhoneAttribute`، `PersonNameAttribute`.
- الميزة اللي التعليق بيوضحها: الرفض بيوصل كخطأ على مستوى الحقل في `ModelState` — مش كرسالة عامة. عشان الفرونت يقدر يحط الرسالة جنب الحقل اللي غلط بالظبط.
- ولاحظ `[PasswordPolicy(EmailProperty = nameof(Email))]` — ال attribute بتوصل لحقل ثاني في نفس ال DTO عشان تتأكد إن كلمة السر مش فيها الإيميل.

فين ده في الكود

`Services/AccountRules.cs`
`AccountRules.ValidatePassword`
`AccountRules.ValidateName`
`AccountRules.ValidateBirthDate`
`PasswordPolicyAttribute`
`Data/Models/DTOS/UserRegistrationDto.cs`

كل القواعد كثنويات ودوال — 258 سطر
الطول والتركييب والكلمات الشائعة
بعد ال trim، ولازم حرف
حساب العمر الحقيقي 18-100
بتربط القاعدة بال DTO وقت ال binding
مكان استعمال ال attributes

اختبر نفسك

- ليه التحقق مكرّر في الفرونت والباكند؟
 - ليه فيه حد أقصى لكلمة السر؟
 - القواعد بتوصل لل endpoint إزاي؟
 - لو غيرت قاعدة في `AccountRules.cs` تعمل إيه؟
- أغيرها في `rules.ts` كمان — قاعدة مكتوبة في `AGENTS.md`.

5. Users and Profiles

`UserController` 552 سطر، وهو الكنترولر اللي بيكمل المصادقة: المصادقة بتخلّي الحساب موجود، وده بيخليه ظاهر ومُدار.

5.1. The Public Profile — One Rule, Two Places

اعرف ده كويس

أهم تعليق في الملف كله:

«المالك والأدمن يشوفوا الرّفّ كله؛ وأي حد ثاني يشوف **اللي هو فعلاً يقدر يوصله**. والعِدّادات تحت لازم تستخدم **نفس الشرط** بتاع الشبكات اللي بتترسم تحتها — ترويسة مكتوب فيها «5 مشاريع» فوق 3 بطاقات دي **كذبة**، وكمان بتقول لغريب إن المؤسّس ده مخبّي كام إدراج أو مستنني مراجعة كام.»

يعني الرقم في الترويسة والبطاقات تحته **لازم** يطلعوا من نفس الشرط لو اختلفوا، مش بس شكل وحش — دي **تسريب معلومات** عن حاجات مخفية.

دي إجابة ممتازة لو الدكتور سأل «ليه اتعبت نفسك تكرر شرط الرؤية؟»

5.2. One Round Trip Instead of Six

اعرف ده كويس

التعليق بيقول: «دورة واحدة للبروفایل وكل عدّاد وعضويات الفريق. دول كانوا **ست** `await` **متتالية** على صفحة قاعدة في نص حلقة التصفّح ← المشروع ← المؤسّس.»

يعني كان فيه ستّ رحلات منفصلة للداتايز، بقوا رحلة واحدة.

ودي أوضح **تحسين أداء** في نصيبك — ولو الدكتور سأل «عملت إيه للأداء؟» دي تيجي مع تخزين الصور المؤقت.

5.3. Account Deletion — Four Guards

`DELETE me` أخطر endpoint في نصيبك، وفيه أربع حمايات:

الحماية	التفصيل
كلمة السر مطلوبة	التعليق بيقول: «لأن دي حاجة مش بيقدر المستخدم يتراجع عنها».
حذف ناعم مش حقيقي	«الصف بيفضل عشان تاريخ حد ثاني ميتهدّش. ده مقصود — التزام المستثمر المعتمد وسجلّ المؤسّس بمين دعمه حقائق مشتركة ، وطرف واحد بيمشي ماينفعش يمسح نسخة الطرف الثاني.»
آخر أدمن محمي	«عمرّك ما تخلّي المنصّة تفقد آخر مشرف عندها: من غير أدمن متبقّي، مفيش حاجة تتراجع ولا بلاغ يتقفل ولا حساب يترجّع.»
مشاريعه بتمشي معاه	«إدراجات المؤسّس بتمشي معاه — وإلا الجمهور هيفضل شايف جولة مفيش حد قادر يجاوب عنها.»

جدول 10 أربع حمايات على إغلاق الحساب.

خد بالك

وبعد كل ده: «إلغاء كل جلسة عشان الحساب ميتستعملش بعد النداء ده.»
يعني الحذف الناعم مش بس بيخفي — بيقل الباب كمان. من غير الخطوة دي، حد معاه توكن صالح كان هيفضل داخل على حساب «متحذف».

5.4. Investor Fields Are Ignored, Not Rejected**اعرف ده كويس**

تفصيلة تصميم واجهة برمجية لطيفة في `PUT me`:

«حقول المستثمر بس. **بتتجاهل بصمت** لغير الأدوار الثانية بدل ما تترفض، عشان المؤسس وهو بيحفظ بروفايله ميتعترض فيها.»
يعني لو مؤسس بعث حقل خاص بالمستثمر، الرد ما بيقاش خطأ — الحقل بيتجاهل وخلاص. القرار ده بيمنع الفرونت من إنه يضطر بيني نموذجين مختلفين تمامًا.

خد بالك

وفيه تحقق صغير وذكي على نطاق الشيك بتاع المستثمر: «نطاق من ناحية واحدة **صالح**؛ نطاق مقلوب **لا**»
يعني «من 10 آلاف» لوحدها مقبولة، و«لحد 50 ألف» لوحدها مقبولة — بس «من 50 ألف لحد 10 آلاف» مرفوضة.

5.5. Trust Signals — You Feed Them, You Don't Compute Them**اعرف ده كويس**

في الملف تعليق بيقول: «مدخلات إشارات الثقة. كل واحدة فيهم حقيقة ظاهرة أصلاً على الصفحة دي؛ الإشارات بس بتحط عليها اسم أمين.»
المدخلات:

- التزكيات اللي اتكتبت عنه كمؤسس — «الإدخال السُمعي الوحيد اللي فيستورا شايلاه واللي مجاش من العضو نفسه.»
- الجولات اللي وصلها لنهاية معلنة.
- عدد الداعمين، محسوب على المشاريع مش الاستثمارات عشان يبقى مميز بطبيعته ويقدر يشيل نفس شرط الرؤية.
- عضويات الفريق المربوطة بمطابقة الإيميل — «أثر رجعي وتلقائي، مش حاجة حد بيوصلها بإيده.»

حساب الإشارات نفسه في `TrustSignals` — بتاع قائد الفريق. إنت بتجمع المدخلات وتعرض النتيجة.

فين ده في الكود

Controllers/UsersController.cs
UserController · GetMe · UpdateMe
UserController · MarkOnboarded
UserController · DeleteMe
UserController · GetPublicProfile

البروفایل والملف العام والصور — 552 سطر
قراءة وتعديل بروفایل المستخدم الحالي
تسجيل إتمام أول تشغيل — `idempotent`
إغلاق الحساب بربع حمايات
الملف العام بشرط رؤية موحد

اختبر نفسك

1. **ليه العدّاد في الترويسة لازم يطابق البطاقات تحته؟**
لو اختلفوا دي كذبة، وكمان بتفضح لغريب كام إدراج مخفي أو مستني مراجعة.
2. **إيه اللي بيمنع مسح الحساب فعليًا؟**
بيانات الطرف الثاني — التزام المستثمر وسجلّ المؤسّس حقائق مشتركة، فالحذف ناعم.
3. **ليه حقول المستثمر بتتجاهل بدل ما تترفض؟**
عشان المؤسّس وهو بيحفظ بروفايله ميتعترش فيها، والفرونيت ميططرش بيني نموذجين.
4. **إشارات الثقة بتتسبب فين؟**
في `TrustSignals` بتاع قائد الفريق — أنا بجمع المدخلات وبعرض النتيجة.

6. Email Delivery

فصل قصير، بس الإيميل هو اللي بيخلّي التسجيل وإعادة التعيين **يشتغلوا** أصلًا.

6.1. The Interface and the Implementation

```
1 public interface IEmailService
2 {
3     Task SendEmailAsync(string toEmail, string subject,
4                         string htmlBody, string textBody);
5 }
```

اعرف ده كويس

لاحظ إن الدالة بتاخذ **نسختين** من الرسالة: `htmlBody` و `textBody`.
السبب: برامج بريد كتير بتقفل الـ HTML، أو بتعرض النص العادي في المعاينة. إرسال النسختين معناها الكود بيتقرا في الحالتين. `MailKitEmailService` بيني الاثنين في نفس الرسالة عن طريق `BodyBuilder`.

6.2. Failure Is Reported, Not Swallowed

اعرف ده كويس

أهم تعليق في مسار الإيميل موجود في `AuthService`:
«القيمة المرجّعة مهمة. الدالة دي كانت بتبلغ كل فشل في تحذير وتسبب اللي بيناديها يجاوب «تم التسجيل بنجاح، من فضلك أكّد إيميلك» مهما حصل — فمزوّد بيرفض المستلم كان بيتنتج رسالة نجاح مبهجة وشخص مستني للأبد على صندوق بريد عمره ما هيستقبل حاجة. فشل التسليم **مش تفصيلة تتسجل**؛ ده الفرق بين حساب حد يقدر يستعمله وحساب مش قادر.»

عشان كده `TrySendVerificationEmailAsync` بترجع `bool` مش `void`.

خد بالك

ولاحظ فرق مستوى التسجيل بين الحالتين:

- فشل إيميل التحقق بيتسجل كـ `LogError` — «مستوى المشغل يراقبه فعلاً»، لأن محدث يقدر يكمل تسجيل من غيره.
- فشل إيميل إعادة التعيين بيتسجل كـ `LogWarning` — أخفّ، لأن المستخدم عنده حساب شغال أصلاً ويقدر يعيد المحاولة.

الفرق ده مقصود، ولو الدكتور سأل عن التعامل مع الأخطاء فهو مثال ممتاز: **مستوى التسجيل بيعكس خطورة الأثر مش نوع الاستثناء.**

فين ده في الكود

```
Services/IEmailService.cs
Services/MailKitEmailService.cs
AuthService.TrySendVerificationEmailAsync
```

العقد — دالة واحدة بنسختين للرسالة
التنفيذ عبر SMTP بـ MailKit
بترجع bool عشان الرسالة تقول الحقيقة

اختبر نفسك

1. ليه الإيميل بيتبع HTML ونص عادي؟
برامج بريد كتير بتقفل الـ HTML أو بتعرض النص في المعاينة — النسختين بيضمنوا إنه يتقرا.
2. ليه دالة إرسال التحقق بترجع `bool`؟
عشان رسالة التسجيل تقول اللي حصل فعلاً — بلع الفشل كان بيتج رسالة نجاح كاذبة.

7. Projects — Your Largest Controller

`ProjectsController` 1180 سطر، وهو اللي بيغذي **صفحة التصفّح** بتاعة عضو الفرونت. الفصل ده أهم فصل بعد المصادقة.

7.1. The One Rule Everything Starts From

في وسط الملف فيه دالة صغيرة بتحكم كل استعلام موجّه للجمهور:

اعرف ده كويس

التعليق بيسمّيها بالطبط: «ال جولات اللي الجمهور يقدر يشوفها **ويقدر لسه يدعمها**: مُراجعة من أدمن، مش موقوفة من المؤسّس، ومش مقفولة.»
وبيكمل: «كل استعلام موجّه للاكتشاف بيبدأ من هنا عشان القواعد ماتفرقش عن بعض أبدًا.»
دي أهم فكرة معمارية في الكنترولر كله: بدل ما كل endpoint يكتب شروط الرؤية بنفسه (ويسهي عن واحد)، فيه **مصدر واحد** للشرط.

الشرط	ليه
<code>ModerationStatus == "Approved"</code>	الأدمن راجع الإدراج — ده تنفيذ قاعدة «مُراجَع لا موثّق».
<code>LifecycleStatus != "Paused"</code>	المؤسّس مسحش الإدراج مؤقتًا.

الشرط	ليه
الجولة مش مقفولة	الشاشات دي كلها بتجاوب على «أقدر أدمع إيه دلوقتي» — جولة خلصت في قائمة عناونها «مشاريع تدور على تمويل» دي إعلان عن حاجة مابقتش موجودة.

جدول 11 الشروط الثلاثة للرؤية العامة.

خد بالك

الجولة المقفولة **ميتختفيش** — بتفضل موجودة بالرابط المباشر وعلى بروفایل المؤسس، **كسجلّ لما حصل**. الفرق بين «مخفي» و«مش في قائمة الاكتشاف» فرق مهم لو الدكتور سأل.

7.2. Visibility Is Viewer-Relative

نفس المشروع بيتعرض بشكل مختلف حسب مين بيص:

```
1 var viewerId = GetOptionalUserId();
2 var isOwner = viewerId.HasValue && viewerId.Value == innovatorId;
3 var isAdmin = User.IsInRole("Admin");
4 var privileged = isOwner || isAdmin;
```

اعرف ده كويس

isOwner **مش** **isAdmin** — دي قاعدة رقم 4 في **AGENTS.md**: «فحص الدور مش فحص الملكية. ضيف الاتنين.» يعني مش كفاية إن المستخدم دوره **Innovator** عشان يعدل مشروع — لازم يكون **OwnerId** بتاع المشروع هو نفسه. مؤسس ثاني دوره **Innovator** برضه، ولازم ميقدرش يلمس مشروع مش بتاعه. لو الدكتور سأل عن الصلاحيات، ابدأ بالجملة دي.

خد بالك

فيه تفصيلة أمنية دقيقة في **details/{projectId}** التعليق بيوثقها: «سببان مختلفان إن الإدراج مش عام، والاتنين معاً لازم يرجعوا 404 لأي حد ثاني. الإيقاف (Paused) كان متنسي: التصقح كان بيخفي المشروع بس الرابط المباشر كان لسه بيرجعه — يعني إيقاف الجولة كان **بيشيلها من القائمة** بس مش بيسحبها زي ما المؤسس متوقع.» ودي نقطة قوية: **الإخفاء من القائمة مش نفس منع الوصول**.

7.3. Search — Two Different Behaviours by Length

أذكرى حتة في البحث، والتعليق بيشرحها:

طول الكلمة	السلوك
قصيرة	بتتطابق على حدود الكلمة بس . من غير ده، «AI» كانت بتطابق C-ai-ro و Dub-ai و N-ai-robi.
طويلة	بتفضل تطابق أي جزء، عشان «health» تفضل تلاقي HealthTech.

جدول 12 البحث بيتصرف حسب طول الكلمة.

اعرف ده كويس

وفيه قرار ثاني مهم: الوصف مستبعد من البحث عن قصد.

التعليق يقول: «التطابق الجزئي جوّه نص طويل خلّى الكلمات القصيرة زي «AI» تطابق كل حاجة تقريبًا من غير ما تقول للمستخدم أي معلومة.»

فالبحث بيشتغل على: الاسم، السطر التعريفي، القطاع، المدينة، والمؤسس.

خد بالك

فيه دالة صغيرة اسمها بتعمل **LIKE** لرموز — يعني % و _ اللي المستخدم يكتبهم يتعاملوا كنص عادي مش كرموز بحث.

من غيرها، حد يدور على «50%» كان هيلاقى كل حاجة، لأن % في SQL معناها «أي شيء». ودي **مش** حقن SQL — ال EF بيمنع ده — بس هي سلوك غلط بيخلّي البحث ميرجّعش اللي المستخدم قصده.

7.4. Facets — Counts That Never Lie

العّدادات جنب كل فلتر، وهي أذكى استعمال في نصيبك:

اعرف ده كويس

التعليق بيشرح القاعدة: «كل facet بيتحسب مع تطبيق كل الفلاتر ما عدا فلتره هو — عشان الأرقام تجاوب على سؤال: هيطلعلي إيه لو اخترت ده بدل اللي مختاره دلوقتي؟»

ودي الفكرة اللي بتمنع الحالة السخيفة: خيار مكتوب جنبه رقم، وأول ما تدوس عليه تلاقي صفر نتيجة.

خد بالك

وفيه تفصيلة ثانية: «Category و Industry قائمة واحدة اسمها «القطاع» بالنسبة للمتصفح؛ ومشروع متسجل في الاثنين ما يتعدّش مرتين.»

يعني حاجة المؤسس بيملاها في خانتين منفصلتين، المتصفح بيشفوها مفهوم واحد — وده قرار تجربة استخدام اتنقذ في الباكند.

7.5. Closing a Round — The Most Careful Code You Wrote

مش بيغيّر عمود واحد — بيعمل عملية كاملة. `POST {projectId}/close-round`

اعرف ده كويس

التعليق بيشرح ليه مفيش طريقة ثانية لإقفال جولة:

«Closed» كانت مقبولة هنا قبل كده، وكانت بتكتب عمود واحد ومفيش حاجة ثانية: مفيش نتيجة، مفيش علاقات اتقفلت، و — الجزء المهم — مفيش سحب لطلبات التمويل اللي لسه معلقة على الجولة. يعني المشروع بيان مقفول والمستثمر لسه بيتطلب منه فلوس. فيه طريقة واحدة لإقفال جولة، وهي اللي بتنصف وراها.»

الحالة	اللي بيحصل لها عند الإقفال
طلب تمويل غير مدفوع	بيتسحب. الجولة خلصت، فمينفعش حد يتطلب منه فلوس ليها.
عملية دفع شغالة دلوقتي	بتتساب في حالها — قطعها وسط الدفع بينتج عملية دفع مالهاش مكان تتعلّق بيه.
علاقة ممولة	مبتترفضش أبدًا مهما كانت حالتها — الفلوس وصلت والسجل لازم يعكس ده.
التزام معتمد	بيفضل بيتحسب في إجمالي الجولة — العلاقة بس بتوصل لنهايتها.

الحالة	الذي يحصل لها عند الإقفال
طلب لسه ما اتوافقش	بيترفض بسبب مذكور — عشان المستثمر ميفضلش مستني جولة مابقتش موجودة.
جدول 13 خمس حالات مختلفة، خمس معالجات مختلفة.	

اعرف ده كويس

القرار الأخلاقي في الكود: التعليق يقول «النتيجة يقولها المؤسس، وماتستنتجش من الأرقام أبدًا. هو بس اللي يعرف إذا كانت جولة وصلت 60% نجاح ولا تخلي، والتخمين هيحط ادعاء في بقه.» دي أقوى إجابة لو الدكتور سأل «ليه مش بتحسبوا النجاح تلقائيًا من النسبة؟»

خد بالك

خد بالك من الحد: الإقفال بيلمس طلبات تمويل وعلاقات استثمار — ودول مفاهيم بتاعة القائد. إنت اللي كاتب ال endpoint، بس المنطق المالي نفسه بيمشي على قواعد FundingMath بتاعة القائد. لو الدكتور سأل في تفاصيل الحساب المالي: «الحساب نفسه في FundingMath بتاع قائد الفريق.»

7.6. Rejected Listings Can Come Back

```
1 // Editing a rejected listing is the founder acting on the reviewer's
2 // note, so it re-enters the queue.
```

اعرف ده كويس

المؤسس لما يعدل مشروع مرفوض، المشروع بيرجع لطاير المراجعة تلقائيًا. التعليق بيوضح المشكلة اللي ده حلها: «من غير ده الرفض كان نهائي: المؤسس يصلح كل حاجة ويفضل عمره ما حد بيصله ثاني.» مثال بسيط وواضح على «الحالة لازم تتحرك في الاتجاهين.»

7.7. Images — Why They're Cached Hard

اعرف ده كويس

التعليق بيشرح: «معرف الصورة ثابت — تغيير الصورة بيعمل صف جديد — فالبايتس ينفع تتخزن مؤقتًا بقوة. من غير ده كل تغيير فلتر كان بيعيد تحميل الشبكة كلها.» وفيه ETag كمان عشان المتصفح ياخذ 304 Not Modified بدل البايتس ثاني. دي نقطة أداء حقيقية في نصيبك، وحلوة لو الدكتور سأل «عملت إيه للأداء؟»

7.8. Soft Delete

خد بالك

حذف المشروع مش حذف حقيقي: «الحذف الناعم: المشروع بيتخفي (بفلتر الاستعلام العام) بس مش بيتشال فعليًا، عشان المؤسس اللي يحذف بالغلط مضيعش نهائيًا.» ودي مرتبطة بشغل عضو الداتايز — هو بيشرح آلية الفلتر العام، إنت بتستخدمها.

```

Controllers/ProjectsController.cs
ProjectsController.GetProjects
ProjectsController.GetFacets
ProjectsController.GetProjectDetails
ProjectsController.CloseRound
ProjectsController.GetProjectImage

```

فمين ده في الكود
أكبر كنترولر عندك — 1180 سطر
قائمة مشاريع مؤسس، برؤية حسب الزائر
العدادات — كل فلتر ما عدا نفسه
تفاصيل مشروع + إشارات الثقة
إفقال الجولة وتنضيف كل اللي معلق
البايتس ب ETag وتخزين مؤقت

اختبر نفسك

1. إيه شروط ظهور المشروع للجمهور؟
مراجع من آدم، مش موقوف من المؤسس، والجولة مش مقفولة — وكل استعمال اكتشاف بيبدأ من نفس الدالة.
2. ليه فحص الملكية مش كفاية بفحص الدور؟
لأن مؤسس ثاني دوره `Innovator` برضه — لازم `OwnerId` يطابق كمان.
3. العدادات بتتحتسب إزاي؟
كل فلتر بيتحتسب مع تطبيق كل الفلاتر ما عدا فلتره هو، عشان الرقم يجاوب «هيطلعلني إيه لو اخترت ده».
4. ليه البحث بيفرق بين الكلمة القصيرة والطويلة؟
القصيرة على حدود الكلمة عشان «Al» ماتطابقش Cairo؛ الطويلة تطابق أي جزء عشان «health» تلاقي HealthTech.
5. ليه إفقال الجولة مش مجرد تغيير عمود؟
لأنه لازم يسحب طلبات التمويل المعلقة كمان — وإلا المشروع بيان مقفول والمستثمر لسه بيتطلب منه فلوس.

8. Story, Signals, and Engagement

باقي نصيبك: القصة والتحديثات، البحث المحفوظ، والتفاعل.

8.1. Project Story — Four Sections in One Controller

`ProjectStoryController` (574 سطر) مقسوم بتعليقات واضحة:

القسم	بيعمل إيه
UPDATES	تحديثات المؤسس — الخط الزمني.
MILESTONES	خارطة الطريق ومراحل المشروع.
TEAM	أعضاء الفريق وصورهم.
DOCUMENTS	غرفة البيانات — المستندات وتحميلها.

جدول 14 أقسام `ProjectStoryController`.

اعرف ده كويس

نشر تحديث بيعت إشعارات لثلاث فئات مختلفة:

1. كل مستثمر عنده استثمار **معتمد**.
 2. المتابعين اللي مش مستثمرين أصلاً.
 3. أي حد حافظ المشروع في قائمة المتابعة — والتعليق يقول عنهم: «اللي حافظ المشروع بيتابعه عن قصد — ودي الفكرة كلها من قائمة المتابعة. دول كانوا المجموعة المهتمة الوحيدة اللي عمرها ما اتقالها إن الحاجة اللي بتتابعها اتحرّكت.»
- ودي إضافة حقيقية لتجربة المستخدم، مش مجرد كود.

خد بالك

الإشعارات دي بتتخطّ في طابور مش بتتبع واحدة واحدة:

«متطابورة عشان نشر التحديث يفضل سريع مهما كان حجم الجمهور، بدل ما يستنّى حلقة حفظ-ودفع متسلسلة لكل مستلم.»

الطابور نفسه (`NotificationFanOutQueue`) بناع القائد — إنت بتخطّ فيه بس.

اعرف ده كويس

وفيه احترام لتفضيلات المستخدم: «تحديثات المشروع محيطة (ambient)، فهي واحدة من الإشعارين اللي العضو يقدر يقلّهم.»

يعني الكود بيتأكد من إعداد المستخدم قبل ما بيعتله.

8.2. A Foreign-Key Detail Worth Knowing

خد بالك

في حذف مستند، التعليق بيشرح حاجة تقنية دقيقة:

«أي طلب مستند كان المستند ده بيجابو عليه **بفتح تاني** بدل ما يختفي معاه — المستثمر لسه محتاج المستند، والطلب هو سجلّ ده. اتعمل صراحةً لأن الـ FK عليه `NoAction` : SQL Server بيرفض `SET NULL` هنا، لأن الجدولين أصلاً بيتسلسلوا من `Projects`.»

دي أعمق تفصيلة قاعدة بيانات في نصيك — ولو الدكتور سأل عنها، الإجابة إن **القيد في الداتايز فرض عليّا أعمل التنضيف يدويًا في الكود.**

8.3. Saved Searches — Computed on Read

`SignalsController` (286 سطر) بيدير البحث المحفوظ.

اعرف ده كويس

التعليق بيشرح الفكرة والقرار المعماري:

«مفيش حاجة في فيستورا كانت بتدي العضو سبب يرجع: كل إشعار كان بيتطلب إنك تكون على الموقع أصلاً عشان تشوفه. البحث المحفوظ بيحوّل استعلام لمرة واحدة لاهتمام دائم، وعدد المطابقات الأحدث من آخر نظرة **بيتحسب وقت القراءة** — فمفيش feed متولّد مسبقًا محتاج يفضل متزامن، ومفيش شارة قديمة.»

ودي إجابة معمارية قوية: **الحساب وقت القراءة بدل التوليد المسبق.** أبسط، ومستحيل يبقى قديم.

8.4. Engagement — Honest Metrics

(325 سطر) فيه ثلاث قرارات كلها عن الأمانة: `ProjectEngagementController`

القرار	التفصيل
المشاهدات مش مضخمة	بتتفلتر لكل زائر في نافذة 30 دقيقة، إعادة تحميل الصفحة متعّدش مشاهدات جديدة.
منحنى «المجموع» بقى منحنين	واحد للالتزامات (الوعود) وواحد للفلوس اللي وصلت فعلاً. التعليق بيقول إن التحليلات كانت بترسم الموافقات وتسمّيها «تم جمعه».
نسبة التحويل أقسى	«بتقيس الزوار اللي مُولوا فعلاً ، مش اللي اتوافق عليهم — نسبة أقسى بكثير وأصدق بكثير».

جدول 15 ثلاثة قرارات في التحليلات، كلها عن الصدق.

اعرف ده كويس

التركيّات بتتعدّ ومبتتحسبش متوسط:

«4 من 5 داعمين قالوا إنهم هيدعموا المؤسّس ده ثاني» — دي جملة **يمكن التحقق منها**. «4.2 نجمة» — دي لأ. ودي واحدة من أوضح تطبيقات فلسفة المنتج كله في كودك: **ماتدّعيش حاجة ماتقدرش تثبتها**.

خد بالك

شروط كتابة تركيبة **انشدّت** لما الفلوس بقت حقيقة — والتعليق بيشرح: «التركيبة تصريح عن إزاي المؤسّس تصرف مع حد **حطّ فلوس**؛ مالهش أي وزن من حد اتوافق عليه ومدفعش أبداً، وجولة جمعت توقعيات ومجمعتش فلوس مادّتش لحد حاجة يحكم عليها». فالشرط: **المستثمر مُول فعلاً** و الجولة مكتملة التمويل.

8.5. Bookmarks and Follows — Idempotent by Design

اعرف ده كويس

كل عمليات الحفظ والمتابعة **idempotent** — يعني نداء العملية مرتين بيدّي نفس النتيجة بالطبط. ليه ده مهم عملياً؟ ضغطة مزدوجة من المستخدم، أو شبكة بتعيد المحاولة، ماينفعش ينتجوا صقّين في الداتايز أو إشعارين لنفس الحاجة.

ولاحظ حاجة صغيرة وذكية في `FollowController`: **ماينفعش تتابع نفسك**.

فين ده في الكود

Controllers/ProjectStoryController.cs
Controllers/SignalsController.cs
Controllers/ProjectEngagementController.cs
Controllers/BookmarkController.cs
Controllers/FollowController.cs
Controllers/FeedController.cs

التحديثات والمراحل والفريق والمستندات
البحث المحفوظ ومركز الإجراءات
المشاهدات والتحليلات والتركيّات
حفظ المشاريع — idempotent
المتابعة — idempotent، ومنع متابعة النفس
تدفق النشاط العام — للادمن فقط

اختبر نفسك

1. مين بياخد إشعار لما المؤسّس ينشر تحديث؟
المستثمرين المعتمدين، المتابعين، واللي حافظين المشروع في قائمة المتابعة.
2. عدد نتائج البحث المحفوظ بيتحسب إمتى؟
وقت القراءة — مفيش feed متولّد مسبقًا، فمفيش شارة قديمة.
3. ليه التركبات بتتعدّ مش بتتسبب متوسط؟
«4 من 5 قالوا كذا» جملة يمكن التحقق منها؛ «4.2 نجمة» لأ.
4. مين يقدر يكتب تزكية؟
المستثمر اللي مؤلّ فعلاً، والجملة لازم تكون مكتملة التمويل.
5. يعني إيه idempotent؟
نداء العملية مرتين بيدي نفس النتيجة — عشان الضغطة المزدوجة ماتعملش صقّين ولا إشعارين.

9. Endpoints Reference — All of Yours

كل نقطة نهاية في نصيبك، بيمين مسموح له يناديها. لو الدكتور سأل «إيه ال API بتاعك؟» — افتح الصفحات دي.

9.1. Auth — `api/auth`

Method + Route	مين	بيعمل إيه
POST register	الكل	تسجيل حساب جديد وإرسال كود تحقق.
POST login	الكل	دخول — بيرجّع access + refresh token.
POST refresh	الكل	تدوير التوكن (بيحتاج refresh token صالح).
POST revoke-token	مسجّل	إلغاء توكن معيّن.
POST logout	مسجّل	إلغاء كل الجلسات النشطة.
POST verify-email	الكل	تأكيد الإيميل بالكود.
POST resend-verification	الكل	إعادة إرسال الكود (بفترة تهدئة).
POST change-password	مسجّل	تغيير كلمة السر + إلغاء الجلسات.
POST forgot-password	الكل	طلب كود إعادة تعيين.
POST reset-password	الكل	إعادة التعيين بالكود.

جدول 16 عشر نقاط للمصادقة.

9.2. Users — `api/users`

Method + Route	مين	بيعمل إيه
GET me	مسجّل	البروفایل الكامل للمستخدم الحالي.
PUT me	مسجّل	تعديل الحقول القابلة للتعديل.
POST me/onboarded	مسجّل	تسجيل إتمام أول تشغيل — idempotent.
DELETE me	مسجّل	إغلاق الحساب (حذف ناعم، بكلمة السر).

Method + Route	مين	يُعمل إيه
GET {id}	الكل	الملف العام + إحصائيات + حالة المتابعة.
GET {id}/backed	الكل	المشاريع اللي دعمها (معتمدة فقط).
GET {id}/cover	الكل	بايتس صورة الغلاف.
GET {id}/avatar	الكل	بايتس الصورة الشخصية.

جدول 17 ثمانى نقاط للمستخدمين.

خد بالك

لاحظ إن الكنترولر كله عليه `[Authorize]` فوق الكلاس، والنقاط العامة بتتفكّ صراحةً بـ `[AllowAnonymous]`.
 ده ترتيب أأمن: الافتراضي إن كل حاجة مقفولة، والفتح بيبقى قرار واعٍ. العكس — الافتراضي مفتوح — معناه إن أي نقطة تتنسى بتبقى مكشوفة.

9.3. Projects — api/projects

Method + Route	مين	يُعمل إيه
GET	الكل	موجز التصفح — بحث وفلاتر وترتيب وصفحات.
GET filters	الكل	قيم الفلاتر للنماذج (بدون عدّادات).
GET facets	الكل	قيم الفلاتر بعدّادات حيّة.
GET details/{projectId}	الكل	تفاصيل مشروع + تعليقات + إشارات.
GET {innovatorId}	حسب الزائر	مشاريع مؤسّس — المالك يشوف الكل.
GET {ownerId}/cards	الكل	مشاريعه كبطاقات للبروفيل.
GET {projectId}/next	الكل	اقتراحات بعد قراءة المشروع.
POST	مسجّل	إنشاء مشروع — بيبداً مخفي لحد المراجعة.
PUT {projectId}	المالك	تعديل — المرفوض بيرجع للطاير.
PATCH {projectId}/lifecycle	Innovator	إيقاف/استئناف الإدراج.
POST {projectId}/close-round	Innovator	إفقال الجولة وتنضيف المعلق.
DELETE {projectId}	المالك	حذف ناعم.
POST {projectId}/comments	مسجّل	تعليق.
POST comments/{id}/replies	مسجّل	ردّ على تعليق.
POST {projectId}/images	المالك	رفع صورة.
GET images/{imageId}	الكل	بايتس الصورة — بـ ETag وكاش.

جدول 18 أهم نقاط المشاريع (مش كلها — فيه زيادة/نقصان عدّادات وحذف).

9.4. Story, Signals, Engagement

Method + Route	مين	يُعمل إيه
GET {projectId}/updates	الكل	الخطّ الزمني للتحديثات.
POST {projectId}/updates	Innovator	نشر تحديث + إشعارات متطابورة.

Method + Route	مين	بيعمل إيه
GET {projectId}/milestones	الكل	خارطة الطريق.
GET {projectId}/team	الكل	أعضاء الفريق.
GET {projectId}/documents	مسجل	قائمة غرفة البيانات.
GET documents/{id}/download	مسجل	تحميل مستند.
GET signals/searches	مسجل	عمليات البحث المحفوظة + العد.
POST signals/searches	مسجل	حفظ بحث جديد.
GET signals/action-center	مسجل	اللي محتاج قرار من المستخدم.
POST {projectId}/view	الكل	تسجيل مشاهدة (مفلتر 30 دقيقة).
GET {projectId}/analytics	Innovator	تحليلات المشروع — للمالك.
GET {projectId}/reviews	الكل	التزيكات (تعدّ ولا تُتوسّط).
POST {projectId}/reviews	Investor	كتابة تزيكية — بشروط أهلية.
POST bookmark/{projectId}	مسجل	حفظ مشروع — idempotent.
POST follow/{userId}	مسجل	متابعة — idempotent، مش نفسك.
GET feed	Admin	تدقّق النشاط العام.

جدول 19 نقاط القصة والإشارات والتفاعل.

10. Request and Response Examples

الأمثلة دي تنفع تفتحها قدام الدكتور مباشرة.

10.1. Register

الطلب:

```

1 POST /api/auth/register
2 {
3   "firstName": "Kazem",
4   "lastName": "Mohamed",
5   "email": "kazem@example.com",
6   "password": "StrongPass99",
7   "confirmPassword": "StrongPass99",
8   "userType": "Investor",
9   "birthDate": "2000-05-14",
10  "phone": "+201234567890"
11 }
```

الرد الناجح — 201:

```

1 {
2   "message": "Registration completed successfully. Please verify your email.",
3   "emailDelivered": true
4 }
```

لو الإيميل ما اتبعثش — لسه 201، برسالة مختلفة:

```
1 {
2   "message": "Your account was created, but we could not send the verification
code...",
3   "emailDelivered": false
4 }
```

اعرف ده كويس

خلّي بالك من `emailDelivered` — ده مش حقل تجميلي. الفرونت بيستخدمه عشان يعرف يوري المستخدم زرار «إعادة إرسال» فورًا ولا لأ.

لو التحقق فشل — 400 بأخطاء على مستوى الحقل:

```
1 {
2   "errors": {
3     "Password": ["Use at least 10 characters."],
4     "Phone":    ["That phone number looks too short."]
5   }
6 }
```

10.2. Login

```
1 POST /api/auth/login
2 { "email": "kazem@example.com", "password": "StrongPass99" }
```

الرد — 200:

```
1 {
2   "message": "Login successful.",
3   "accessToken": "eyJhbGciOiJIUzI1NiIs... ",
4   "expiresAt": "2026-08-13T15:42:00Z",
5   "refreshToken": "9xK2mP-vQ8...",
6   "refreshTokenExpiresAt": "2026-08-27T14:42:00Z",
7   "userId": 42,
8   "userType": "Investor",
9   "userName": "Kazem Mohamed",
10  "userEmail": "kazem@example.com",
11  "hasOnboarded": false
12 }
```

اعرف ده كويس

`hasOnboarded` بييجي من `user.OnboardedAtUtc != null` — الفرونت بيستخدمه عشان يقرّر يوديه على الأونبوردنج ولا على لوحته مباشرة.

شاشة الأونبوردنج نفسها بتاعة قائد الفريق — إنت بتوفر الحالة اللي بتحدّد تظهر ولا لأ.

10.3. Browse Feed

```
1 GET /api/projects?search=health&sector=HealthTech&sort=momentum&page=1
```

الرد بيبقى صفحة نتائج، وكل بطاقة فيها الأرقام المالية جاية من `FundingMath` بتاعة القائد.

10.4. A Facet Response

```

1 GET /api/projects/facets?sector=HealthTech
2
3 {
4   "sectors": [ { "value": "HealthTech", "count": 12 },
5                 { "value": "MedTech", "count": 4 } ],
6   "locations": [ { "value": "Cairo", "count": 7 } ]
7 }

```

اعرف ده كويس

لاحظ: العدّادات محسوبة وفلتر القطاع متطبّق على الباقي بس. عشان كده `MedTech` جنبها 4 — ودي إجابة هيطلعلي كام لو غيّرت اختياري لـ `MedTech`.

11. Authorization — Role Is Not Ownership

الفصل ده قصير بس هو أكثر حاجة الدكتور بيحب يسأل فيها.

11.1. The Rule

اعرف ده كويس

قاعدة رقم 4 في `AGENTS.md` بالنص: «فحص الدور مش فحص الملكية. ضيف الاتنين.»

- `[Authorize]` بيتأكد إن فيه حد مسجّل دخول.
- `[Authorize(Roles = "Innovator")]` بيتأكد إن دوره مؤسس.
- **ولا واحد فيهم** بيتأكد إن المشروع ده بتاعه هو.

الفحص التالت لازم يتكتب بإيدك في الكود:

```

1 if (project.OwnerId != GetCurrentUserId())
2   return Forbid();

```

11.2. The Three Levels in Your Code

المستوى	مثال من كودك
مفتوح للكل	<code>[AllowAnonymous]</code> — الملف العام، بايتس الصور، موجز التصفح.
أي مستخدم مسجّل	<code>[Authorize]</code> — الحفظ، المتابعة، التعليق.
دور معيّن	<code>[Authorize(Roles = "Innovator")]</code> — نشر تحديث، إقفال جولة. و <code>Roles = "Investor"</code> لكتابة تركية. و <code>Roles = "Admin"</code> للتدقّق.
دور + ملكية	تعديل مشروع: الدور من الـ <code>attribute</code> ، والملكية بفحص <code>OwnerId</code> جوة الدالة.

جدول 20 أربع مستويات صلاحية في كودك.

خد بالك

ولاحظ إن الرؤية نفسها ستتغير حسب الزائر مش بس السماح:

```
1 var privileged = isOwner || isAdmin;
```

المالك ببشوف مشاريعه في كل الحالات، والزائر ببشوف المراجع وغير الموقوف بس. يعني نفس الـ endpoint بيرجع بيانات مختلفة لناس مختلفة — وده مقصود ومكتوب في تعليق فوق الدالة.

اعرف ده كويس

فيه مثال ممتاز على الحماية في `DELETE me`:

التعليق بيقول: «عمرك ما تخلي المنصة تفقد آخر مشرف عندها: من غير أدمن متبقي، مفيش حاجة تتراجع، ولا بلاغ يتقفل، ولا حساب يترجع.»

يعني الكود بيرفض إن آخر أدمن يقفل حسابه. حماية من حالة مستحيل الرجوع منها.

اختبر نفسك

1. إيه الفرق بين فحص الدور وفحص الملكية؟

الدور بيقول إنت إيه، الملكية بتقول الحاجة دي بتاعتك ولا لأ. مؤسس ثاني دوره `Innovator` برضه.

2. ليه الكنترولر كله `[Authorize]` والنقاط العامة `[AllowAnonymous]`؟

عشان الافتراضي يبقى مقفول — أي نقطة تتنسى تبقى محمية مش مكشوفة.

3. نفس الـ endpoint بيرجع بيانات مختلفة ليه؟

الرؤية viewer-relative: المالك والأدمن يشوفوا الكل، وغيرهم يشوف المراجع وغير الموقوف.

4. إيه اللي بيمنع آخر أدمن يقفل حسابه؟

فحص صريح في `DELETE me` — من غير أدمن مفيش مراجعة ولا بلاغات تتقفل.

12. The Database Boundary

الفصل ده قصير بس لازم تحفظه، لأنه أسهل سؤال ممكن تقع فيه.

12.1. You Consume the Schema, You Don't Change It

اعرف ده كويس

لو الدكتور سأل: «لما احتجت عمود جديد في الداتابيز عملت إيه؟»

الإجابة: «يطلب من قائد الفريق يعمل الـ entity والـ migration. أنا بستهلك السكيما وبكتب الاستعلامات

عليها، بس مش أنا اللي بغيرها.»

ودي مش نقطة ضعف — دي انضباط. الداتابيز مشتركة وبعيدة، وتغيير السكيما من كذا ناحية في نفس الوقت أسرع طريقة لكسر شغل الكل.

12.2. What You Do Touch

بنعمله	مثال من كودك
استعلامات LINQ	<code>_dbContext.Projects.Where(...).Select(...)</code> — ده شغللك كله تقريبًا.
الحفظ	<code>_dbContext.Users.Add(user)</code> ثم <code>.SaveChangesAsync()</code>
التحميل الصريح	<code>.Include(u => u.RefreshTokens)</code> لما تحتاج علاقة مربوطة.
تجاوز التتبع	قراءات العرض محتاجش تتبّع تغييرات.

جدول 21 إنت بتكتب استعلامات، مش سكيما.

خد بالك

فيه تفصيلة واحدة **الداتايز فرضتها على كودك** — وذكرناها في فصل القصة: ال FK على طلبات المستندات عليه `NoAction`، و SQL Server بيرفض `SET NULL` هناك لأن الجدولين أصلًا بيتسلسلوا من `Projects`. فالكود بتاعك لازم ينصّف الطلبات يدويًا عند حذف المستند. دي أوضح مثال على إن **قيود السكيما بتغير شكل الكود** — حتى لو مش إنت اللي كاتبها.

اعرف ده كويس

والحذف الناعم كمان: `DELETE {projectId}` و `DELETE me` بيعملوا **حذف ناعم** مش حقيقي، لأن فيه فلتر استعلام عام (global query filter) في `AppDbContext` بيخفي المحذوف تلقائيًا. يعني إنت بتغير عمود، والإخفاء بيحصل لوحده. آلية الفلتر نفسها بتاعة عضو الداتايز.

اختبر نفسك

1. **بتغير السكيما إزاي؟**
مبغيرهاش — بطلب من القائد يعمل ال entity وال migration.
2. **ليه ده انضباط مش ضعف؟**
الداتايز مشتركة، وتغيير السكيما من كذا ناحية أسرع طريقة لكسر شغل الفريق كله.
3. **الحذف الناعم بيشتغل إزاي في كودك؟**
بغير عمود بس، والفلتر العام في `AppDbContext` بيخفي الصف تلقائيًا.

13. Known Limitations in Your Area

الأمانة هنا أقوى من التجميل. لو اتسألت «إيه العيوب؟» — دي إجابتك.

الحدّ	التفصيل
البحث بـ <code>LIKE</code>	مش محرّك بحث حقيقي (زي full-text search). كفاية للحجم الحالي، بس مش هيفضل كفاية لو المحتوى كبير أضعاف.
الصور في الداتايز	البايتس متخزنة في القاعدة نفسها مش في تخزين خارجي. مفايزة مقصودة — القرار وشرحه عند عضو الداتايز.
التحقق مكرّر يدويًا	<code>AccountRules.cs</code> و <code>rules.ts</code> بيتزامنوا بالانضباط ، مش بأداة. نسيان تحديث واحدة منهم مظهرش غير وقت التشغيل.

الحدّ	التفصيل
الأدوار جوّه التوكن	تغيير دور مستخدم مش بيسري غير لما التوكن القديم ينتهي.
قائمة كلمات السر الشائعة	ثابتة في الكود — مش قائمة محدّثة من مصدر خارجي.
جدول 22 حدود معروفة في نصيبك.	

اعرف ده كويس

الطريقة الصح تقول بيها أي حدّ من دول:

«ده قرار واعٍ للنطاق الحالي، مش حاجة فاتتنا. البديل كان يكلف [كذا]، والحجم الحالي ما يستاهلش.»
الفرق بين الطالب اللي عارف حدود شغله والطالب اللي مش شايفها = الفرق في التقدير.

14. Doctor Questions and Pressure Follow-ups

كل سؤال ومعاها أسئلة المتابعة اللي ممكن تيجي لو ضغط.

مثال

«إيه دورك في الباكند؟»

الإجابة: «المصادقة والحسابات كاملة، والمشاريع والاكتشاف والقصة والتفاعل — 9 كترولرز و4 خدمات، حوالي 4675 سطر.»

لو ضغط: «طيب الدفع؟» — «الدفع والتمويل بتوع قائد الفريق: `PaymentService` و `FundingMath`.»

لو ضغط: «وغرفة الصفقة؟» — «برضه قائد الفريق — `DealRoomController`.»

مثال

«إزاي بتخزن كلمة السر؟»

الإجابة: «هاش بـ Bcrypt — `HashPassword` عند الحفظ و `Verify` عند الدخول. كلمة السر نفسها مبيتخزنش أبدًا.»

لو ضغط: «ليه Bcrypt مش SHA256؟» — «SHA سريع، وده عيب هنا: بيخلي تجربة ملايين الاحتمالات رخيصة. Bcrypt بطيء عن قصد.»

لو ضغط: «وليه حائط حدّ أقصى 72 بايت؟» — «لأن Bcrypt بيتجاهل أي حاجة بعد 72 بايت. لو قبلنا أطول، هنبقى بنخزن جزء من كلمة السر بصمت، وأي نص ثاني بنفس أول 72 بايت هيفتح الحساب.»

مثال

«ليه رسالة الدخول الغلط عامة؟»

الإجابة: «عشان محدش يعرف مين مسجل عندنا. لو قلت الإيميل مش موجود، أي حد يقدر يجرب قائمة إيميلات ويعرف مين عضو — تسريب مستخدمين.»

لو ضغط: «طيب ليه القفل والإيقاف رسايلهم مختلفة؟» — «لأن دول لحساب إحنا متأكدين إنه موجود أصلاً. إخفاء السبب هناك كان هيتعب المستخدم الحقيقي من غير أي مكسب أمني.»

مثال

«اشرح لي ال refresh token»

الإجابة: «التوكن بيتخزن مهشّر، وكل استعمال بيولد توكن جديد ويلغي القديم — ده اسمه rotation.»

لو ضغط: «طيب لو حد سرق توكن؟» — «هنا بيشتغل كشف إعادة الاستخدام: لو جه توكن مُلغى، معناها فيه نسخة ثانية بتتستعمل، فبنلغي كل جلسات الحساب. المستخدم بيدخل ثاني والسارق بيفضل بلا حاجة.»

لو ضغط: «وليه مخزنه مهشّر؟» — «لو الداتا بيز اتسربت، التوكنات نفسها مش موجودة فيها — نفس منطق كلمة السر.»

مثال

«التحقق مكرّر في الفرونت والباكند — مش ده تكرار؟»

الإجابة: «الفرونت راحة استخدام عشان المستخدم يعرف قبل ما بيعت؛ الباكند هو اللي بيقرّر. أي حد يقدر بيعت طلب ب Postman من غير ما يفتح الصفحة أصلاً.»

لو ضغط: «وإزاي بتضمن إنهم متطابقين؟» — «بالانضباط، والقاعدة مكتوبة في AGENTS.md . وده حدّ معروف: مفيش أداة بتفرض التوافق تلقائيًا.»

مثال

«العدادات جنب الفلاتر بتتسبب إزاي؟»

الإجابة: «كل فلتر بيتسبب مع تطبيق كل الفلاتر الثانية ما عدا فلتره هو — عشان الرقم يجاوب على: هيطلعي إيه لو اخترت ده بدل اللي مختاره؟»

لو ضغط: «وليه ده مهم؟» — «عشان ميحصلش إن خيار مكتوب جنبه رقم وأول ما تدوس عليه تلاقي صفر نتيجة.»

لو ضغط: «والقطاع؟» — «Industry و Category حاجتين منفصلتين للمؤسّس، بس قائمة واحدة للمتصفّح — ومشروع مسجّل في الاثنين مايتعدّش مرتين.»

مثال

«ليه إقفال الجولة معقد كده؟»

الإجابة: «لأنه مش تغيير عمود. لازم يسحب كل طلبات التمويل المعلقة، ويسبب عملية دفع شغالة دلوقتي في حالها، ويرفض الطلبات اللي ما اتوافقتش، ويسبب الممولة زي ما هي.»

لو ضغط: «وليه سايب الدفع الشغال؟» — «قطعه وسط العملية بينج عملية دفع مالهاش حاجة تتعلّق بيها.»

لو ضغط: «ليه المؤسّس يقول النتيجة مش النظام يحسبها؟» — «هو بس اللي يعرف لو جولة وصلت 60% كانت نجاح ولا تخلي. التخمين هيحطّ ادعاء في بقّه.»

مثال

«فحص الصلاحيات عامل إزاي؟»

الإجابة: «تلات مستويات: [AllowAnonymous] للعام، [Authorize] لأي مسجّل، [Authorize(Roles = ...)] لدور معيّن — وفوقهم فحص الملكية يدوي في الكود.»

لو ضغط: «ليه محتاج الملكية طالما فاحص الدور؟» — «لأن مؤسّس ثاني دوره Innovator برضه. الدور

بيقول إنت إيه، الملكية بتقول الحاجة دي بتاعتك.»

مثال

«إيه أصعب حاجة عملتها؟»

الإجابة: «كشف إعادة استخدام ال refresh token. مش صعب في الكود، صعب في التفكير: إن توكن مُلغى بيتستعمل مش خطأ عادي — دي إشارة سرقة، والرد الصح إني ألغي كل الجلسات مش الطلب ده بس.»

مثال

«إيه العيوب في شغلك؟»
 الإجابة: «البحث بـ `LIKE` مش محرّك بحث حقيقي — كفاية للحجم ده مش لأضعافه. والتحقق متزامن بين الفرونت والباكند بالانضباط مش بأداة. وتغيير دور المستخدم مبيسرّيش غير لما التوكن ينتهي.»
 لو ضغط: «وليه ما عملتوش الأفضل من الأول؟» — «كل واحدة فيهم كانت هتكلف وقت وتعقيد أكثر مما الحجم الحالي يستاهل. دي قرارات واعية للنطاق، مش حاجات فاتت.»

15. How to Demo Your Part Live

لو الدكتور قال «ورّيني» — ده السيناريو اللي تمشي عليه.

15.1. Before You Start

```
1 cd MyAppApi/MyAppApi
2 dotnet run
```

خد بالك

لو رفض يشتغل وقال حاجة عن مفتاح Stripe — ده مقصود. المشروع بيرفض يشتغل على مفتاح مش تجريبي، ولازم يبدأ بـ `sk_test_`.
 ودي حماية بتاعة قائد الفريق، بس اعرفها عشان متترعّيش قدام اللجنة لو حصلت.

بعدها افتح Swagger على `/swagger` — كل ال endpoints بتاعتك هناك جاهزة للتجربة من غير أي أداة تانية.

15.2. The Five-Minute Scenario

الخطوة	اللي تعمله وتقله
1. تسجيل	<code>POST /api/auth/register</code> ببيانات صحيحة. وريه الرد وقول: «لاحظ <code>emailDelivered</code> » — الرسالة بتقول اللي حصل فعلاً.
2. كلمة سر ضعيفة	كرّر التسجيل بكلمة سر <code>12345</code> . وريه الخطأ على مستوى الحقل وقول إنه جاي من <code>AccountRules</code> المطابقة للفرونت.
3. دخول	<code>POST /api/auth/login</code> وريه ال <code>accessToken</code> وال <code>refreshToken</code> و <code>hasOnboarded</code> .
4. دخول غلط	نفس الإيميل بكلمة سر غلط، وبعدها إيميل مش موجود خالص . وريه إن الرسالتين متطابقتين — وده منع تسريب المستخدمين.
5. تدوير التوكن	<code>POST /api/auth/refresh</code> بال <code>refresh token</code> . خد الجديد، وبعدين جرّب القديم تاني — وريه إنه اترفض وإن كل الجلسات اتلغت.
6. التصفّح	<code>GET /api/projects?search=health</code> . وريه البحث شغّال.
7. العدّادات	<code>GET /api/projects/facets</code> . قول: «كل رقم محسوب مع كل الفلاتر ما عدا فلتره هو.»
8. الملكية	حاول تعدّل مشروع مش بتاعك بتوكن حساب تاني — وريه الرفض وقول: «الدور مش الملكية.»

جدول 23 ثمان خطوات بتغطّي أقوى حاجات في نصيبك.

اعرف ده كويس

أقوى لحظة في العرض كله هي الخطوة 5. تدوير التوكن وكشف إعادة الاستخدام حاجة قليل ما تتشاف في مشاريع تخرّج — ولما توربها **شغالة** قدامهم، الفرق بين «قرأت عنها» و«نقدتها» بيان فوراً. اتمرّن عليها لحد ما تعملها من غير ما تفكر.

16. Terms You Must Know

المصطلحات اللي ممكن تتسأل فيها مباشرة.

المصطلح	المعنى
ORM	طبقة بتحوّل الكلاسات لجداول والعكس — هنا EF Core.
DbContext	بوابة التعامل مع الداتابيز، وهو نفسه Repository و Unit of Work.
LINQ	لغة الاستعلام اللي بتكتب في C# وبتحوّل لـ SQL.
DTO	كائن نقل بيانات — شكل الطلب أو الرد، منفصل عن كيان الداتابيز.
Model binding	تحويل الـ JSON الجاي لكائن C#، والتحقق بيحصل وقتها.
ModelState	نتيجة التحقق — بتحمل الأخطاء على مستوى كل حقل.
DI	حقن التبعية — بتطلب الحاجة كـ interface والـ container بيجيها.
Middleware	حلقات بتشتغل على كل طلب قبل ما يوصل للكنترولر.
JWT	توكن موقع بيحمل هوية المستخدم ودوره جواه.
Claim	معلومة جوه التوكن — زي NameIdentifier و Role.
Access token	توكن قصير المدة بيتبع مع كل طلب.
Refresh token	توكن طويل المدة بيتبدّل بواحد جديد — وبيتخزن مهشّر.
Token rotation	كل استعمال بيولّد توكن جديد ويلغي القديم.
Reuse detection	استعمال توكن مُلغى = إشارة سرقة ⇒ إلغاء كل الجلسات.
BCrypt	خوارزمية هاش لكلمات السر، بطيئة عن قصد .
User enumeration	تسريب معلومة إن إيميل معيّن مسجّل عندنا ولا لأ.
Idempotent	نداء العملية مرتين بيدّي نفس النتيجة بالطبط.
Soft delete	إخفاء الصف بعمود بدل مسحه فعلياً.
Global query filter	شرط بيتطبّق تلقائياً على كل استعمال لكيان معيّن.
Facet	قيمة فلتر ومعاها عدد النتائج المتوقع.
ETag	بصمة للمحتوى بتخلّي المتصفح ياخذ 304 بدل البايتس تاني.

جدول 24 واحد وعشرون مصطلح.

17. Suggested Study Plan

الجلسة	تقرا	تعمل على الجهاز
الأولى	الفصل 1 و 2 — دورك وحدودك، وإزاي الباكند متوصّل	شغّل الـ API واعمل <code>POST /api/auth/register</code> من Swagger.
الثانية	الفصل 3 — المصادقة	افتح <code>AuthService.cs</code> واتبع <code>LoginAsync</code> سطر سطر.
الثالثة	الفصل 3 ثاني — التوكينات	سجّل دخول، خذ الـ refresh token، دوّره، وجرب تستعمل القديم ثاني.
الرابعة	الفصل 4 و 5 — القواعد والمشاريع	جرب كلمة سر ضعيفة وشوف شكل الخطأ، وافتح <code>/api/projects/facets</code> .
الخامسة	الفصل 6 و 7 — القصة والتفاعل، وجدول الـ endpoints	افتح كل endpoint في نصيبك من Swagger مرة واحدة على الأقل.
السادسة	الفصل 8 و 9 و 10 — الأمثلة والصلاحيات وحدّ الداتايز	جرب تعدّل مشروع مش بتاعك وشوف الرد.
الأخيرة	الفصل 11 و 12 — الحدود وأسئلة الضغط	جواب بصوت عالي من غير ما تبصّ، وبعدها الـ Cheat Sheet.

جدول 25 سبع جلسات، كل واحدة قراءة + حاجة بإيدك.

اعرف ده كويس

أهم نصيحة: **مذاكرش بالقراءة بس**. افتح Swagger وجرب كل endpoint بنفسك. الدكتور مش بيسأل «إيه تعريف الـ JWT» — بيسأل «وريني» و«ليه».

18. Final Code Map — Where to Find Everything

مرجع واحد يجمع كل حاجة انتشرت في الملف. لو الدكتور سأل «فين الكود بتاع الحاجة دي؟» — افتح الصفحة دي بس.

What it does	Where in Code	What
· إنشاء الحساب، هاش كلمة السر، كود التحقق، وإرسال الإيميل.	<code>Services/AuthService.cs</code> <code>RegisterAsync</code>	Registration
· البوابات الخمسة بالترتيب، وإصدار التوكينات.	<code>Services/AuthService.cs</code> <code>LoginAsync</code>	Login
· الـ claims، ومنها الدور اللي بتشتغل بيه <code>[Authorize(Roles=...)]</code> .	<code>Services/AuthService.cs</code> <code>CreateAccessToken</code>	JWT issuing
· تدوير التوكن وكشف إعادة الاستخدام.	<code>Services/AuthService.cs</code> <code>RefreshTokenAsync</code>	Refresh rotation

What it does	Where in Code	What
· عدّ المحاولات، القفل، والفكّ الذاتي.	Services/AuthService.cs	Lockout
·	RegisterFailedLogin	
	IsLockedOut	
· كل حدث أمني بيتكتب في SecurityLogs	Services/AuthService.cs	Security log
	AddSecurityEvent	
· كود مؤقت، فترة تهدئة، وحدّ للمحاولات.	Services/AuthService.cs	Password reset
·	ForgotPasswordAsync	
	ResetPasswordAsync	
· كل قواعد التحقق — مطابقة لـ rules.ts	Services/AccountRules.cs	Account rules
· بتربط القواعد بال DTO وقت ال model binding.	Services/AccountRules.cs	Validation attributes
	PasswordPolicyAttribute	· وإخوته
· عشر نقاط رفيعة بتنده AuthService	Controllers/AuthController.cs	Auth endpoints
· البروفايل، الملف العام، الصور، وإغلاق الحساب.	Controllers/UsersController.cs	Profile
· البحث والفلاتر والترتيب والصفحات.	Controllers/ProjectsController.cs	Browse feed
	GetProjects	
· عدّادات حيّة — كل فلتر ما عدا نفسه.	Controllers/ProjectsController.cs	Facets
	GetFacets	
· التفاصيل + التعليقات + إشارات الثقة.	Controllers/ProjectsController.cs	Project details
	GetProjectDetails	
· إقفال الجولة وتنضيف كل اللي معلّق.	Controllers/ProjectsController.cs	Close round
	CloseRound	
· بايتس الصورة بـ ETag وتخزين مؤقت.	Controllers/ProjectsController.cs	Project images
	GetProjectImage	
· الخطّ الزمني، خارطة الطريق، الفريق، والمستندات.	Controllers/ProjectStoryController.cs	Updates and milestones
· البحث المحفوظ، والعدّ بينحسب وقت القراءة.	Controllers/SignalsController.cs	Saved searches
· إحصائيات المشاركة، منحنيات التمويل، والتزكيات.	Controllers/ProjectEngagementController.cs	Analytics and reviews
· حفظ المشاريع — idempotent.	Controllers/BookmarkController.cs	Bookmarks
· المتابعة — idempotent، ومنع متابعة النفس.	Controllers/FollowController.cs	Follows

What it does	Where in Code	What
تدقق النشاط العام — للأدمن فقط.	Controllers/ FeedController.cs	Activity feed
إرسال رسائل التحقق وإعادة التعيين عبر SMTP.	Services/ MailKitEmailService.cs	Email delivery

جدول 26 كل حاجة في نصيبك، في جدول واحد.

19. Cheat Sheet — Review Before You Walk In

خلاصة

نصيبك: المصادقة والحسابات · المستخدمين والبروفایل · المشاريع والاكتشاف · القصة والتحديثات · البحث المحفوظ · التفاعل والمتابعة · الإيميل. 9 كترولرز و4 خدمات ≈ 4675 سطر.

مش نصيبك: الدفع · غرفة الصفقة · اللوحات · الرسائل والإشعارات · الأدمن · البنية التحتية · الكيانات والترجيلات. كلها قائد الفريق.

الشكل: الكترولر بيكلم `AppDbContext` مباشرة — مفيش Repository. الخدمة بتوجد لما يبقى فيه منطق مشترك حقيقي.

كلمة السر: Bcrypt، وحد أقصى 72 بايت لأن Bcrypt بيتجاهل اللي بعدها.

الدخول: خمس بوابات. البوابة 1 و 4 بيرجّعوا نفس الرسالة — منع تسريب المستخدمين.

التوكن: مهشّر في الداتابيز، بيتدور كل استعمال. توكن مُلغى بيتستعمل = سرقة ⇒ إلغاء كل الجلسات.

القفل: بيفك نفسه في `IsLockedOut` — مفيش مهمة خلفية.

الصلاحية: الدور ≠ الملكية. `[Authorize(Roles=...)]` + فحص `OwnerId`.

الرؤية: مُراجع · مش موقوف · مش مقفول — ومصدرها دالة واحدة مشتركة.

العذادات: كل فلتر بيتحسب مع كل الفلاتر ما عدا فلتره هو.

البحث: الكلمة القصيرة على حدود الكلمة، الطويلة تطابق أي جزء، والوصف مستبعد عن قصد.

الإفعال: مش عمود — بيسحب المعلّق، يسحب الدفع الشّغال، ويحترم الممول. والنتيجة يقولها **المؤسّس**.

التزكيات: تُعدّ ولا تُتوسّط — «4 من 5» قابلة للتحقق، «4.2 نجمة» لأ.

الداتابيز: بتستهلكها مش بتغيّرهما — ال entity وال migration عند القائد.

`AccountRules` ↔ `rules.ts`: مطابقين بالانضباط — القاعدة في `AGENTS.md`.

لو اتسألت في حاجة مش بتاعتك: «دي بتاعة قائد الفريق» — إجابة محترمة.